

Assessment Task

2: Programming for the Web

William Lam

Table of Contents

Functionality Improvements:	2
Text Input Field	2
Edit Entries	3
Delete Entries	4
Search/Filter Function	5
Time Tracker	5
Clear All Entries	6
Cancel/Reset Button	6
Login Page	7
Notification	7
Form Validation and Error Checking	8
Compatibility Across Browsers	12
Vendor Prefixes	12
Testing Browsers	13
Vivaldi	13
Microsoft Edge	14
Firefox	14
Compatibility Across Devices	15
Installing PWA on Smartphone	16
Security Measures	18
Cross-Site Scripting	18
Secure Local Storage Practises	18
Hashing	19
Secure Password Policy	20
OSWASP Zap Scan	21
Risks	21
Load Time Management	23
File Optimisation	23
Lazy Loading	23
Caching Strategies	24
Security by Design	26
Reflection	27

Functionality Improvements:

Text Input Field

Enter the date, description and start & end times

Date

Start time

End time

Description

This text input field is used for adding a description to the sessions. It has a max length of 20 characters (set in the HTML), and if that limit is reached, no new input is allowed.

Once the form is submitted, the description is displayed along with the date, start time and end time, and the buttons.

Past sessions

Total Hours: 29h 9m

01/02/2021 from 11:02 AM to 11:00 PM testing	11h 58m	Edit	Delete
01/02/2020 from 11:11 AM to 2:22 PM asdddddddddddddd	3h 11m	Edit	Delete
10/11/1021 from 11:00 AM to 12:00 PM asd	1 hr	Edit	Delete
10/02/20 from 10:00 AM to 11:00 PM asd	13 hr	Edit	Delete

This allows the user to identify what happened during the session.

Edit Entries

The screenshot shows a web interface for editing session entries. At the top, there are three buttons: '3h 11m', 'Edit', and 'Delete'. The 'Edit' button is circled in red. Below this is a form titled 'Enter the date, description and start & end times'. The form contains four input fields: 'Date' (01/02/2021), 'Start time' (11:02 AM), 'End time' (11:00 PM), and 'Description' (testing). Below the form are four buttons: 'Confirm', 'Cancel', 'Clear All', and 'Open Search'. At the bottom, there is a section titled 'Past sessions' showing 'Total Hours: 29h 9m' and a list of sessions. The first session is '01/02/2021 from 11:02 AM to 11:00 PM testing'. Red arrows and a line connect the 'Edit' button to the 'Date' field, the 'Start time' field, the 'End time' field, the 'Description' field, and the first session entry in the 'Past sessions' list.

3h 11m Edit Delete

Enter the date, description and start & end times

Date 01/02/2021

Start time 11:02 AM

End time 11:00 PM

Description testing

Confirm Cancel Clear All

Open Search

Past sessions

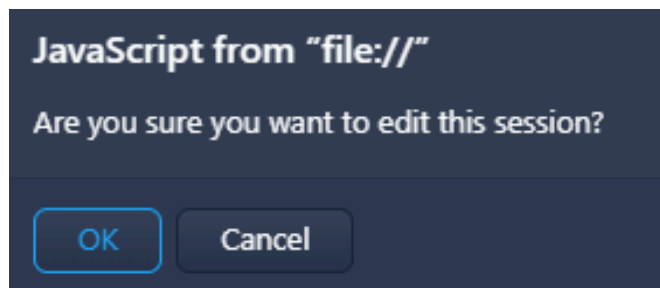
Total Hours: 29h 9m

01/02/2021 from 11:02 AM to 11:00 PM testing

For each session, a button is created. When that button is clicked it triggers the `editSession()` function with the selected session data (date, starttime, endtime, description). It then finds the session in the array of sessions and fills in the original input fields with the selected session data. It also triggers a global variable, which is used to track whether or not a session is being edited. The "Add Session" button has its text changed to "Confirm".

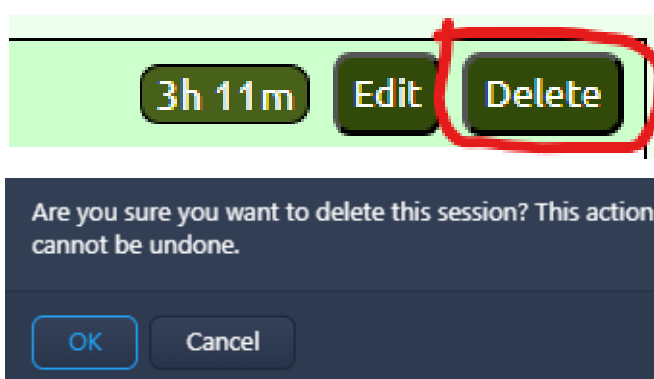
The user can then change the data. Once they click submit, it follows the normal submission process to validate the date, start-time, end-time and description and asks the user to confirm the change through an alert.

If confirmed, it deletes the old session data using the 'Delete Entries' Function and stores the updated data into local storage. It then rerenders all the sessions, which allows the edited session to be sorted and displayed.



This allows the user to edit sessions after they have been submitted, in case they need to fine tune or correct details.

Delete Entries



This function is used to delete entries. For each session, a button with text "Delete" is created. A confirmation will popup.

If confirmed, it calls the `deleteSession()` function with the selected session data (date, start-time, end-time, description).

It then searches the session array for the session that matches the selected session. It then removes it from the `STORAGE_KEY` which is a string constant which stores all the sessions. This deletes the session from local storage. It then rerenders the sessions to show the change, with the deleted session gone.

This allows the user to delete sessions if necessary.

Search/Filter Function

The screenshot shows a search and filter interface. At the top, there are three buttons: 'Add Session', 'Cancel', and 'Clear All'. Below them is a button labeled 'Open Search', which is circled in red. Underneath is a 'Close Search' button. The main section is titled 'Search and Filters'. It contains a 'Date' field with a placeholder 'dd/mm/yyyy' and a calendar icon. Below that are 'Start Time' and 'End Time' fields, each with a placeholder '--:-- --' and a clock icon. There is a 'Description' text input field with the placeholder text 'Weather, Location etc...'. Below the description field are two dropdown menus labeled 'Filter by:' and 'Order by:', both with 'Select' as the current selection. To the right of these dropdowns is a 'Confirm' button. At the bottom left of the search section is a 'Reset' button.

The search/filter form originally starts hidden. It can be toggled by clicking the button with text "Open Search". When it is clicked it changes the text to "Close Search".

For the search, there are four input fields correlating to Date, Start Time, End Time and Description. When data is input, it calls the search() function which first checks what input fields are in use. Not all are required for a search to start. If the user may only want to search by date and start time, and hence only put in values for that. It then creates a const called filteredSessions which

is the filtered results using the search criteria. It calls renderSessions with criteria filteredSessions.

For the filter function, there are two dropdowns. The user must choose two filters and then click "Confirm". Once the button is clicked, it checks whether the filters are actually selected or not (gives alert saying filters are not chosen if one or more is missing). If filters are correct, it then sorts the sessions accordingly. It then calls the renderSessions function with the sorted sessions. That renders the sessions in the filter order.

The reset button will clear both the search field and the filters.

This allows the user to search specifically for sessions. This is especially useful with a large number of sessions.

Time Tracker

The screenshot shows a time tracker interface. At the top, there is a green bar containing a session duration '3h 11m', which is circled in red, and two buttons: 'Edit' and 'Delete'. Below this bar is a section titled 'Past sessions'. Under the title is a summary line 'Total Hours: 17h 11m', which is also circled in red. Below the summary is a list of sessions, each on a new line. Each session entry consists of a date, a time range, and a description. The sessions listed are: '01/02/2020 from 11:11 AM to 2:22 PM asd', '10/11/1021 from 11:00 AM to 12:00 PM asd', and '10/02/20 from 10:00 AM to 11:00 PM asd'.

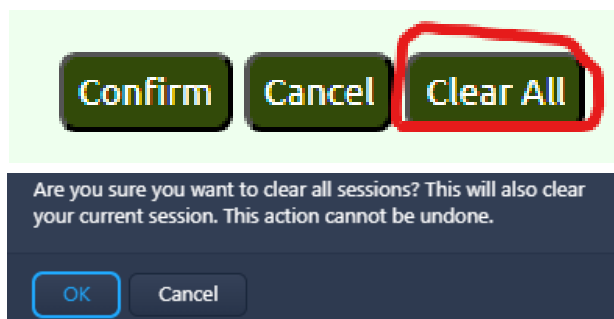
For each session, there is a time tracker that displays the session duration. Additionally, directly underneath the Past sessions header there is a line which shows the total amount of time. For the session tracker, calling calculateSessionHours(), it takes the start time and end time and splits them

into hours and minutes. It then converts each into minutes and subtracts start time from end time. This gets the total session time. It then converts it into hours and minutes and displays the time next to the buttons.

For the total hours, the `calculateTotalHours()` function is called. It goes through every session and calculates the session hours using the `calculateSessionHours()` function. It adds each session's hours to the variable `totalMinutes`. It then displays the total time in hours and minutes.

This is a quality of life feature helping the user to easily see their time, and hence their progress.

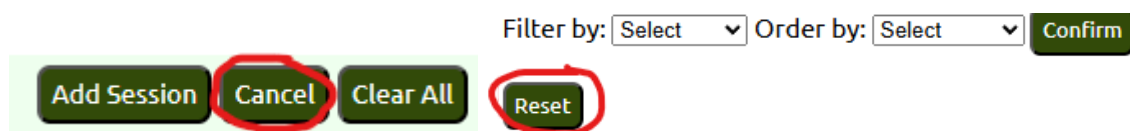
Clear All Entries



This button clears all stored sessions. Upon clicking, the `clearAll()` function is called. It shows a confirmation message. If confirmed, it then resets the session form, then deletes the `STORAGE_KEY`, which effectively deletes all sessions. It then renders to update the change.

This allows users to perform a mass reset at once, if necessary.

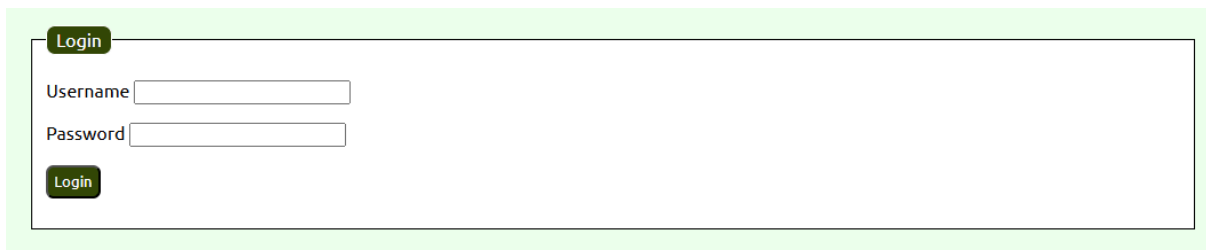
Cancel/Reset Button



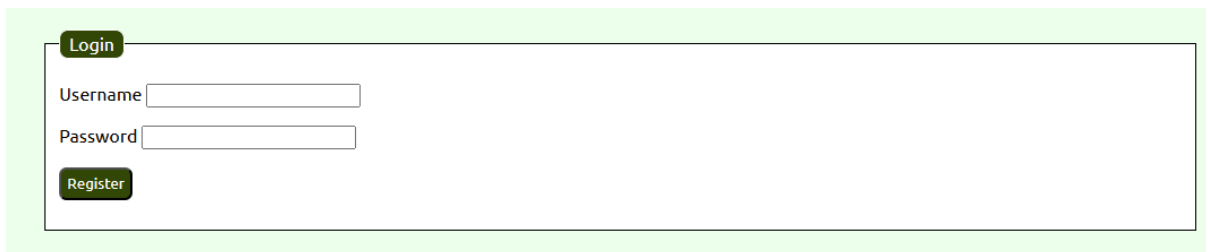
If cancel is clicked, it resets the form, deleting all the current inputted data. It is a button with the type `'reset'`, which allows it to reset the form it is attached to. There is a "Cancel" button for the session form and "Reset" button for the filters/search.

This serves as a quick way to undo any recent changes before accidentally submitting.

Login Page

A login form with a light green background. At the top left is a dark green button labeled 'Login'. Below it are two input fields: 'Username' and 'Password'. At the bottom left is another dark green button labeled 'Login'.

The login page asks for username and password. If it is the first time the user is opening the PWA, it will show:

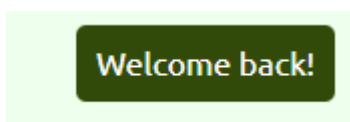
A login form with a light green background. At the top left is a dark green button labeled 'Login'. Below it are two input fields: 'Username' and 'Password'. At the bottom left is a dark green button labeled 'Register'.

The user types in a username and password. Upon clicking register, the password is checked under validation rules to ensure it has at least 8 characters, 1 capital letter and 1 number. If it is valid it then hashes the password and stores it to LOGIN_KEY. It then logs the user into the app and hides the login form, showing the rest of the app as well.

Every time the app is loaded, a login is required. It checks if a user is registered. It allows a maximum of one user. When logging in, the input password is hashed and compared with the stored hash to see if it is the same. If it is the same it allows access to the rest of the app. When clicking the logout button it returns to the login form and hides the rest of the page.

This provides basic security for the app, ensuring the data is safe.

Notification



To replace alerts, a function called `showNotification()` is used, which displays text for a short amount of time before disappearing. It is shown in the bottom right of the screen.

This serves as a quality of life feature allowing information to be passed to users without disrupting the app experience.

Form Validation and Error Checking

Enter the date, description and start & end times

Date

dd/mm/yyyy

!

Please fill in this field.

End time

--:-- --

Description

Weather, Location etc...

All the session inputs are required, which means that if they are empty, it pops up with a message asking users to complete it. Additionally, only certain types of data can go into each input.

Date: dd/mm/yyyy

Start Time: HH:MM:AM/PM

End Time: HH:MM:AM/PM

Description: string

Upon submission, all data is checked using functions to be both accurate and logical.

checkDateInvalid()

JavaScript

```
function checkDateInvalid(date) {
  // Check that date is not null.
  if (!date) {
    // as date is invalid, we return true
    return true;
  }

  //checks if date is in future
  const today = new Date(); //gets todays date
  today.setHours(0, 0, 0, 0); //sets time to 00:00 to compare only date
  const inputDate = new Date(date); //gets input date as date object
  if (inputDate > today) {
    showNotification("Future dates are not allowed.");
    return true;
  }

  // else
  return false;
}
```

```
}
```

This function validates the date. If empty, it exits the function. If not empty, it then gets the current date, setting the time to midnight so that only the dates are compared. It turns the input date into a proper date object and then compares the two. If the input date is in the future, it displays a notification and exits, not submitting.

checkTimesInvalid()

JavaScript

```
function checkTimesInvalid(startTime, endTime, date) {

    const sessions = getAllStoredSessions();
    //checks if times overlap with existing session on the same day
    for (const session of sessions) {
        if (session.date === date) { //only checks sessions on the same day
            //skip the session being edited to allow changes
            if (editing && session.date === editingSession.date &&
                session.startTime === editingSession.startTime
                && session.endTime === editingSession.endTime && session.desc ===
                editingSession.desc) {
                continue;
            }
            //checks if the new session overlaps with the existing session
            if (startTime === session.startTime && endTime === session.endTime) {
                showNotification("Session already exists with the same start and end
time.");
                return true;
            }
            //checks if new sesh starts before existing sesh ends, and if new sesh
            ends after existing sesh starts
            if (startTime < session.endTime && endTime > session.startTime) {
                showNotification("Time overlaps with existing session.");
                return true;
            }
        }
    }

    // Check that end time is after start time and neither is null.

    if (!startTime || !endTime || startTime > endTime) {
        showNotification("Time is invalid");
        // as times are invalid, we return true
        return true;
    }
}
```

```
// else
return false;
}
```

This function checks the validity of the start and end times. It first checks if the dates are the same with an existing session. If they are, it then checks if the start time or end time overlaps with that existing session's time. If it does, the data is invalid. If it does not, then the function checks if the start time is before the end time, to ensure it logically makes sense. If invalid, it shows a notification.

sanitiseInput()

```
JavaScript
//sanitisation function to prevent XSS attacks
function sanitiseInput(input) {
    return DOMPurify.sanitize(input);
}
```

This function is used to sanitise input, usually the description fields. DOMPurify is a third party library imported and the function `sanitize()` is used to clean the input to ensure an XSS attack cannot occur.

When submitting/editing a session, after the description is sanitised it is checked to see if it is empty via:

```
JavaScript
if (!desc.trim()) { //removes whitespace and checks if description is empty
    showNotification("Description is invalid or empty after sanitisation.
Please enter valid text.");
    return;
}
```

`.trim` removes all whitespace from the description. If it is empty it shows a notification and does not submit the session.

showNotification()

JavaScript

```
function showNotification(message) {  
  const notification = document.getElementById('notification');  
  notification.textContent = message;  
  notification.classList.remove('hidden');  
  setTimeout(() => {  
    notification.classList.add('hidden');  
  }, 3000); //disappears after 3 seconds  
}
```

This function serves as a replacement for alerts. It finds the div called 'notification' in the HTML file, setting it as a constant variable. The criteria used when calling the function is then set as the text Content, and the notification is shown for three seconds. It can be used to display error messages, as well as other information the user may find necessary.

checkPasswordStrength()

JavaScript

```
function checkPasswordStrength(password) {  
  //checks if password is at least 8 characters, contains a number, an  
  //uppercase letter, and a lowercase letter  
  const regex = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}$/;  
  return regex.test(password); //returns true if all requirements are met,  
  //false otherwise  
}
```

When the function is called, it creates a constant called regex which stores the validation rules which are:

- At least 8 characters long .{8,}
- At least 1 number (?=.*\d)
- At least one lowercase letter (?=.*[a-z])
- At least one capital letter (?=.*[A-Z])

It then tests the password against the criteria.

Compatibility Across Browsers

Vendor Prefixes

Examples:

```
CSS
#buttonContainer {
  /* normal css */
  -webkit-box-pack: end;
  -ms-flex-pack: end;
  -webkit-box-align: center;
  -ms-flex-align: center;
}
```

```
CSS
/*mobile responsiveness*/
@media screen and (max-width: 450px) {
  #buttonsContainer {
    /* normal css */
    -webkit-box-orient: vertical;
    -webkit-box-direction: normal;
    flex-direction: column;
    -webkit-box-align: stretch;
    -ms-flex-align: stretch;
  }
}
```

AutoPrefixer, a modern tool for developers, was used to add compatibility between a range of different browsers quickly. It adds vendor prefixes (e.g. webkit) automatically, saving time. Vendor prefixes are used because not every browser supports the exact same core CSS.

AutoPrefixer: <https://autoprefixer.github.io>

Testing Browsers

Method of Testing

For each browser, the following steps were taken:

1. Register a username and password
 - a. Put in bad password to see if it flags it
2. Ensure that CSS formats the PWA correctly.
3. Add sessions and check that validation works
4. Test search and filter by inputting values and using the dropdowns
5. Test if sessions are able to be edited
6. Test if sessions are able to be deleted
7. Test logout and login verification
8. Test clearAll sessions

Computer Specs used for testing:

Name: HP Z4G4

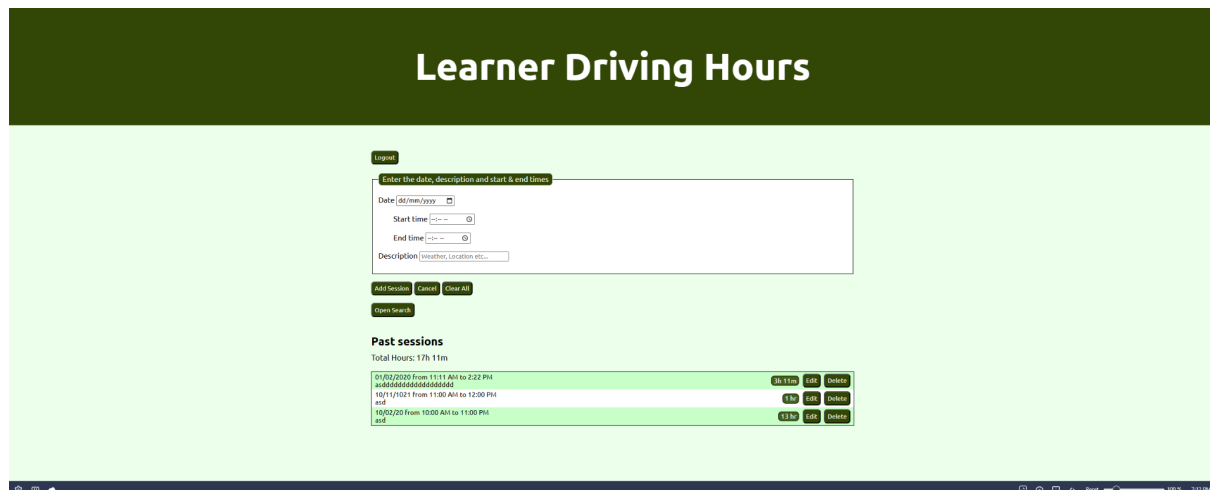
CPU: Intel Xeon W-2145

RAM: 128GB DDR4 ECC 2666MT/S

GPU: RX 5700XT

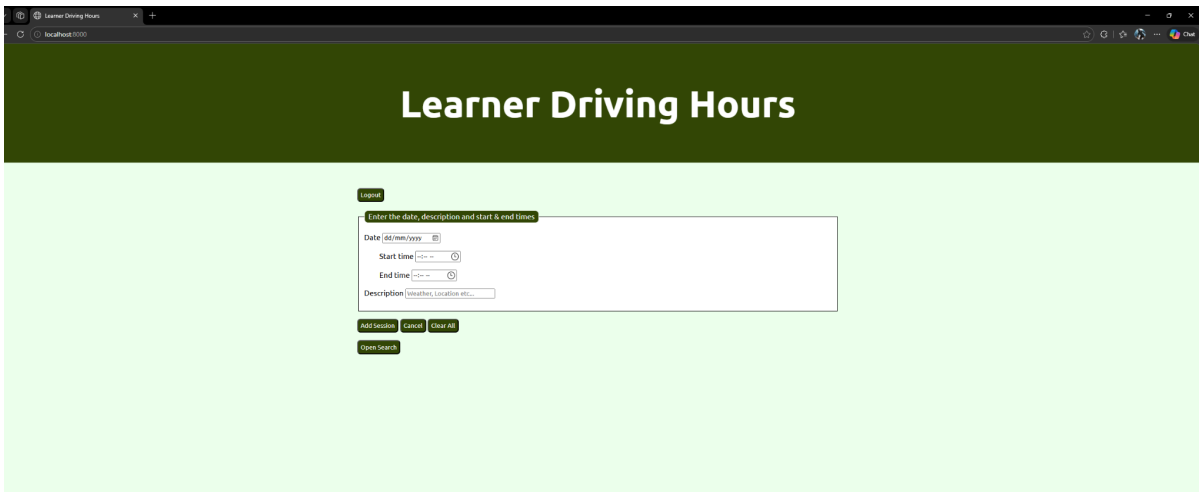
OS: Windows 11 Pro for Workstations 25H2

Vivaldi



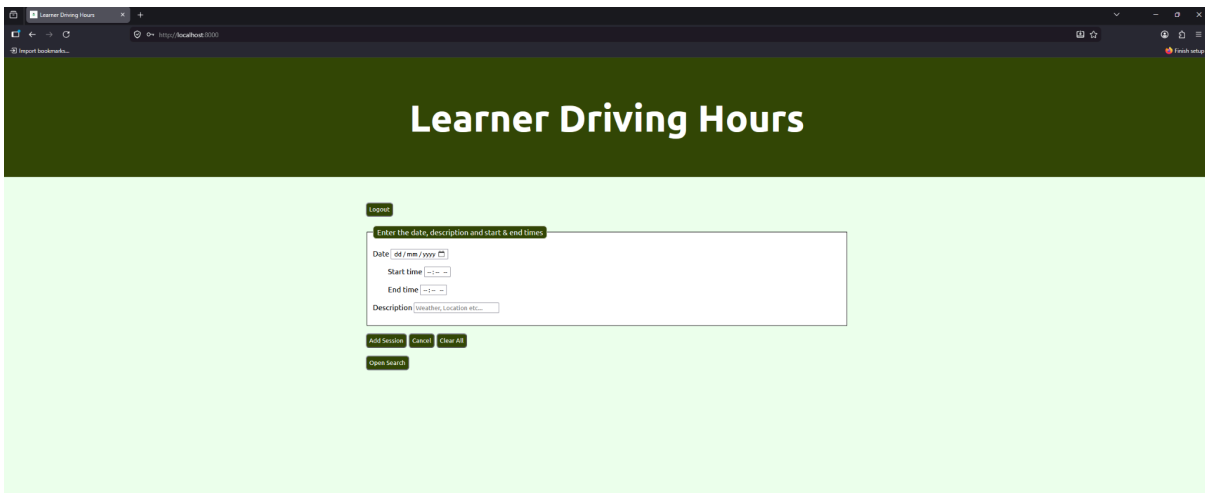
Vivaldi passed all tests.

Microsoft Edge



Microsoft Edge passed all tests.

Firefox



Firefox passed all tests.

Compatibility Across Devices

Mobile View

CSS

```
/*mobile responsiveness*/
@media screen and (max-width: 450px) {
  /*adjust body width for smaller screens*/
  body {
    width: 90%;
  }
  h1 {
    font-size: 1em;
  }
  li {
    white-space: normal;
    -ms-flex-wrap: wrap;
    flex-wrap: wrap;
  }
  #buttonsContainer {
    -webkit-box-orient: vertical;
    -webkit-box-direction: normal;
    -ms-flex-direction: column;
    flex-direction: column;
    -webkit-box-align: stretch;
    -ms-flex-align: stretch;
    align-items: stretch; /* Makes buttons full width of the container */
    gap: 0.5em /* Adds space between buttons when stacked vertically */
  }
  /* Make buttons full width on small screens */
  button {
    width: 100%;
    margin: 0.5em; /* Adjust padding for better touch targets */
  }
}
```

Tablet View

CSS

```
/*tablet responsiveness*/
@media screen and (min-width: 451px) and (max-width: 1000px) {
  body {
    width: 70%;
  }
  h1 {
    font-size: 2em;
  }
}
```

```

li {
  white-space: normal;
  -ms-flex-wrap: wrap;
  flex-wrap: wrap;
}
#buttonsContainer {
  -webkit-box-orient: vertical;
  -webkit-box-direction: normal;
  -ms-flex-direction: column;
  flex-direction: column;
  -webkit-box-align: stretch;
  -ms-flex-align: stretch;
  align-items: stretch; /* Makes buttons full width of the container */
  gap: 0.5em /* Adds space between buttons when stacked vertically */
}
}

```

To adapt the layout of the website for devices, @media screen was used, with a minimum and maximum width set in pixels to indicate different devices. Within @media, the CSS can be rewritten for different sizes. If something is formatted outside of @media, but is not changed in @media, it remains at the same formatting despite the change in screen size.

Additionally, relative units (em) instead of px were used so that sizes would scale according to the size of the screen.

Installing PWA on Smartphone

Process:

From Set 13.06

Host PWA on computer

1. Find folder of PWA and copy directory
2. Open Command Prompt and type "cd directory"
3. Run command: python -m http.server
4. Server will then be hosted on the default port of 8000

Access PWA on smartphone

(Following steps are for Vivaldi Mobile on Android - but steps should be similar for all browsers)

1. Find IPV4 on hosting computer (ipconfig on cmd)
2. Type in on smartphone browser: IPVY:PORT
 - a. E.g. 192.168.0.1:8000
3. Find browser menu, usually three dots or similar
4. Search for "add page to" or similar

5. Click "add to home screen" or similar
6. May be asked to name PWA: "Learner Hours"
7. Drag shortcut to homescreen

Note: given that this is a HTTP PWA not HTTPS, the service worker does not save. To solve this, at least for chromium based browsers, you will need to edit the flags to allow insecure connections to be trusted. This allows the PWA to work offline.

Source:

<https://www.chromium.org/Home/chromium-security/deprecating-powerful-features-on-insecure-origins/>

Security Measures

Cross-Site Scripting

Cross-Site Scripting (XSS) is a vulnerability where malicious scripts, often JavaScript, are injected into websites. The browser can then execute that code, allowing attackers to steal vital data such as session tokens, cookies, or even write HTML content. It allows attackers to essentially imitate an unsuspecting user performing actions for them.

The method I used to prevent this was Input Sanitisation through a third party library called DOMPurify.

```
HTML
<!-- gets DOMPurify -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.0.6/purify.min.js">
</script>
```

sanitiseInput()

```
JavaScript
//sanitisation function to prevent XSS attacks
function sanitiseInput(input) {
    return DOMPurify.sanitize(input);
}
```

I import DOMPurify in the HTML and then create the sanitiseInput() function which uses the DOMPurify.sanitize function. This sanitises the input to prevent XSS and then returns to wherever it was called from.

I used this mostly to sanitise my description input fields.

Additionally to prevent XSS without a third party library, I used textContent which automatically encodes any input as plain text.

Secure Local Storage Practises

- What are secure local storage practices and how do they assist in security?
How were they implemented in your solution? Include code snippets.
- Explain the secure password policy that you implemented and how you implemented it. Include code snippets.

Secure local storage practises ensure that data directly on the client side is kept secure and free from exposure. They are a set of rules that provide detail on how to protect your data when storing it.

They aid in security through preventing XSS attacks, providing encryption, data isolation, and input validation. They reduce the risk of an incident, as well as the overall scale.

Hashing

In my PWA, I used a hashing algorithm to protect my user's password. I implemented SHA-256, turning the password into a 64 character hexadecimal string.

hashPassword()

JavaScript

```
//hashing function for password
async function hashPassword(password) {
  const encoder = new TextEncoder();
  const data = encoder.encode(password);
  //specifies hash used (SHA-256) and hashes it
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  const hashHex = hashArray.map(b => b.toString(16).padStart(2,
'0')).join('');
  return hashHex;
}
```

The constant encoder is a TextEncoder which turns the password into raw bytes through the constant data. It then performs the hash function by providing the algorithm to use (SHA-256). This returns as an ArrayBuffer which is then converted into a normal array through the constant hashArray.

It then creates the hexadecimal string and returns it.

This prevents the raw password from being saved to localStorage, which is very easy for attackers to retrieve through the console. This makes it harder for attackers to get the password.

Secure Password Policy

Below is my password policy.

checkPasswordStrength()

JavaScript

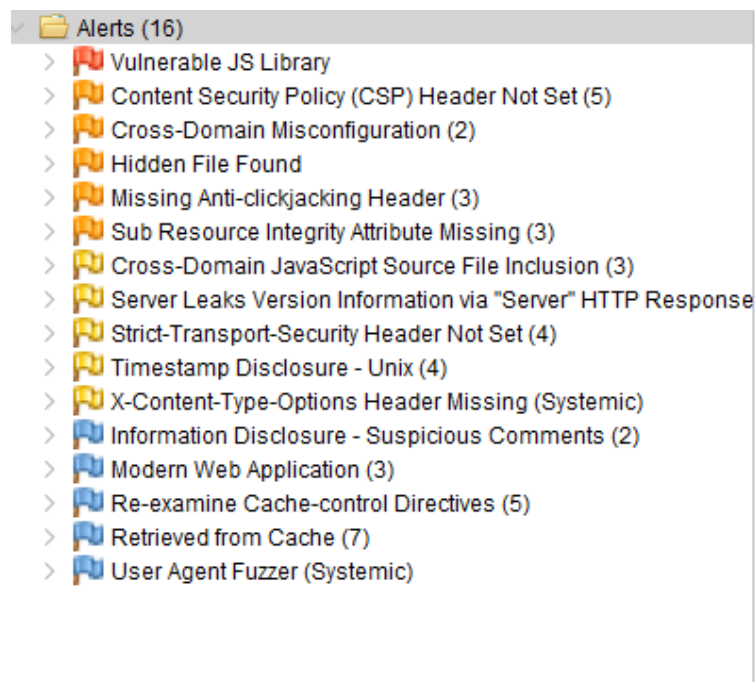
```
function checkPasswordStrength(password) {  
    //checks if password is at least 8 characters, contains a number, an  
    uppercase letter, and a lowercase letter  
    const regex = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}$/;  
    return regex.test(password); //returns true if all requirements are met,  
    false otherwise  
}
```

When the function is called, it creates a constant called regex which stores the validation rules which are:

- At least 8 characters long `.{8,}`
- At least 1 number `(?=.*\d)`
- At least one lowercase letter `(?=.*[a-z])`
- At least one capital letter `(?=.*[A-Z])`

It then tests the password against the criteria. It must meet all these requirements before it is valid.

OSWASP Zap Scan



Risks

1. Vulnerable JS Library

- a. Risk: High
- b. Explanation: The identified library appears to be vulnerable.
- c. Solution: Upgrade to the latest version of the affected library.

2. Cross-Domain Misconfiguration

- a. Risk: Medium
- b. Explanation: Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.
- c. Solution: Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance). Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

3. Hidden File Found

- a. Risk: Medium
- b. Explanation: A sensitive file was identified as accessible or available. This may leak administrative, configuration, or credential information which can be leveraged by a malicious individual to further attack the system or conduct social engineering efforts.
- c. Solution: Consider whether or not the component is actually required in production, if it isn't then disable it. If it is then ensure

access to it requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.

All information taken directly from the Alert details on OSWASP.

Load Time Management

File Optimisation

In order to optimise my CSS and JS files, I used an online minifier to remove all unnecessary whitespace, putting the entire file on one line. It also changes variables names to be simpler, for example 'a' or 'b'.

For [app.js](#) and [sw.js](#) I used Terser: <https://try.terser.org>

For style.css I used Toptal: <https://www.toptal.com/developers/cssminifier>

This would decrease overall load time, especially for large files, but keep the same functionality.

Note: Un-minified files will be uploaded as well for reference given that the minified files are unreadable (called originalname_large e.g. sw_large.js)

Lazy Loading

Lazy loading is a performance optimisation strategy that delays loading of non critical resources until they are needed, which is often done through scrolling or navigation of the page.

I implemented lazy loading on the rendering of my sessions.

JavaScript

```
//limits max number of sessions at a time (lazy loading)
let sessionsShown = 10;
function renderLazySessions() {
  const allSessions = getAllStoredSessions();
  //only take a slice of the array to reduce no. of sesh
  const visibleSessions = allSessions.slice(0, sessionsShown);

  renderPastSessions(visibleSessions);

  //adds a button to load more if there are more sessions left
  if (sessionsShown < allSessions.length) {
    const loadMoreBtn = document.createElement("button");
    loadMoreBtn.innerText = "Load More";
    loadMoreBtn.onclick = () => {
      sessionsShown += 10;
      renderLazySessions();
    };
    pastSessionContainer.appendChild(loadMoreBtn);
  }
}
```

```
//search function
function search() {
  visibleSessions = 10; //resets number to 10 seshs
  //rest of search function

}
```

It sets a variable of sessionsShown to 10. Then in renderLazySessions() it slices the sessions array to reduce the number of sessions to the sessionsShown. It then renders that amount of sessions.

If there are more sessions left, it creates a button that allows the user to manually load more, ten at a time. Additionally, as the user scrolls down more sessions will load.

This would be very useful if the user had a large amount of stored sessions. It also does the same to the search function, so that the rerendering of searched sessions can cause lag due to sheer volume (except it does not create a button).

Caching Strategies

Efficient caching strategies improve PWA performance by reducing load and latency.

The original [sw.js](#) used a Cache First strategy which is good for speed and has capable offline capabilities, but means that the user will never see new updates unless the cache is manually refreshed or invalidated. I also ran into issues where my PWA would not load when updating my service worker.

Chosen Strategy: Cache First with Network Fallback

I reworked the original caching strategy of Cache First, adding a network fallback instead of a 404 error. By doing this, I can reduce scenarios where the PWA won't be working, increasing overall reliability. It also solved the error I was running into where the service worker wouldn't update properly.

```
JavaScript
//attempts to use cache first, then network
self.addEventListener("fetch", (event) => {
  //only get static files (GET requests means retrieving data, not modifying it)
  if (event.request.method !== "GET") return;

  event.respondWith(
    (async () => {
```

```

const cache = await caches.open(CACHE_NAME);

//verify cache
const cachedResponse = await cache.match(event.request);

if (cachedResponse) {
  //if found in cache, return it immediately
  return cachedResponse;
}

//if not in cache, go to network
try {
  const networkResponse = await fetch(event.request);

  //store in cache if valid response for future use
  if (networkResponse.ok) {
    cache.put(event.request, networkResponse.clone());
  }

  return networkResponse;
} catch (error) {
  //if all fail, give fallback (e.g. may be offline)
  return new Response("Offline content not available", { status: 503
});
}
})();
);
});

```

First, it filters the requests to only be GET, which are used for retrieving static data like index.html. This prevents dynamic data like forms being retrieved, which service workers are not equipped for.

It then attempts to use its cache first. It verifies it, and if it is there it loads immediately without going to the network. However if that fails it attempts to go the network to retrieve a new cache. If both failed, it will give error 503 which is "Service Unavailable". Causes can be the device or server being offline, or service being overloaded.

Security by Design

Security by Design is where something is built with security as a core function, instead of an afterthought. It is directly within every stage of the Software Development Life Cycle (SDLC).

This allows the software to be molded around security and risks are clearly identified and dealt with.

Security Features in my PWA:

- Authentication
 - Login Page
 - Hashing of Passwords
- Input Validation
 - Use of DOMPurify to sanitise
 - Use `textContent` to encode directly to strings
- Default State
 - Upon every reload, the user is immediately brought back to the login screen, which helps it default to a 'locked' state.

In the development of this PWA the key principles, such as Principle of Least Privilege (PoLP), Defence in Depth, and Fail Securely could have been applied.

PoLP

Ensures that users and modules have the minimum permissions required to perform their functions.

Defence in Depth

Using multiple security measures. Encryption could have been used alongside hashing.

Fail Securely

If a system crash, reset or error is to occur, it should default to a 'locked' state, ie no sensitive information is immediately displayed.

Overall this would vastly improve the security of the PWA and reduce the risks of cyberattacks.

Reflection

Throughout this task, I have vastly improved my knowledge of how websites work, and how to code in HTML, CSS and JavaScript. Prior to this assessment, my understanding of these languages was fairly surface level, however it has now pushed me to further my knowledge much more. JavaScript was definitely the one I learnt most about, given its complexity compared to HTML and CSS. I developed a much stronger grasp of core concepts such as functions, event listeners, and DOM manipulation, as well as more advanced topics like service workers and caching strategies. The [app.js](#) was definitely the file that took the longest to complete. I am happy that I was able to come up with my own improvements alongside the ones given, which have improved my critical thinking skills.

My understanding of how PWAs work have vastly changed over the course of the task. They are vastly different to ordinary websites. Securing the PWA was a tough challenge. Protecting against XSS was difficult, especially before I learnt the hash functions. Even despite that, I feel like the PWA is not properly secure. However I am glad that I was able to import and use a third party library, which I feel will be a useful skill to have.

Time management was another major challenge throughout the task. I lost almost half of the time due to my exam block, and then I had another massive hand-in assessment due the day before this one. I had to work tirelessly for a couple of nights to finish it on time. I can only hope my interview goes well at this point.

I was able to apply knowledge to make my PWA compatible with different devices through @media, and was able to get it working with different browsers as well. I was also able to research into caching strategies and was able to make a slight improvement to the service worker, which I am glad to have done.

In the future, I would like to build the PWA with Security by Design, as it was an afterthought in this task. I would like to use PBKDF2 alongside SHA-256 to implement a better hashing function using salts as SHA-256 by itself can be brute forced quite easily. Unfortunately I am out of time to do that. I would also like to code a proper backend, including a secure database. This would allow me to practise PoLP and Defense in Depth. I would also like to be able to use HTTPS instead of HTTP, which provides more security and would allow me to properly host the PWA.

In conclusion, this task has majorly improved my skills as a software developer in the languages of HTML, CSS and JS. Given my ambitions to go into the tech industry, perhaps under cybersecurity, having a solid foundation of knowledge of Security by Design will aid immensely. I am glad that I was able to finish on time, but will need to work on my time management.